



01: Learning from data and related challenges and linear models for regression

220472T-Perera P.L,P

1 Question 1

```
1
2
3
4 #install required libs
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.preprocessing import StandardScaler, MinMaxScaler,
  ↪ MaxAbsScaler
8
9 x = np.array([0,1,2,3,4,5,6,7,8,9])
10 y_true = np.array([20.26, 5.61, 3.14, -30.00, -40.00, -8.13,
  ↪ -11.73, -16.08, -19.95, -24.03])
11
12 x_1 = np.array([0,9])
13 slope , intercept = np.polyfit(x, y_true, 1) #degree 1 for linear
  ↪ fit
14 y_1 = slope * x_1 + intercept
15
16 print(f"Slope: {slope :.2f}")
17 print(f"Intercept:{intercept :.2f}")
18
19 plt.figure(figsize=(10, 6))
20 plt.scatter(x, y_true, color='blue', label='Data points')
21 plt.plot(x_1, y_1, color='red', label='Model')
22 plt.xlabel('x')
23 plt.ylabel('y')
24 plt.title('Scatter Plot with Linear Regression Models')
25 plt.legend()
26 plt.grid(True)
27 plt.show()
28
```

1.2 Scatter plot of data points with linear regression line

The computed slope and intercept of regression line are:

Slope: -3.56

Intercept: 3.92

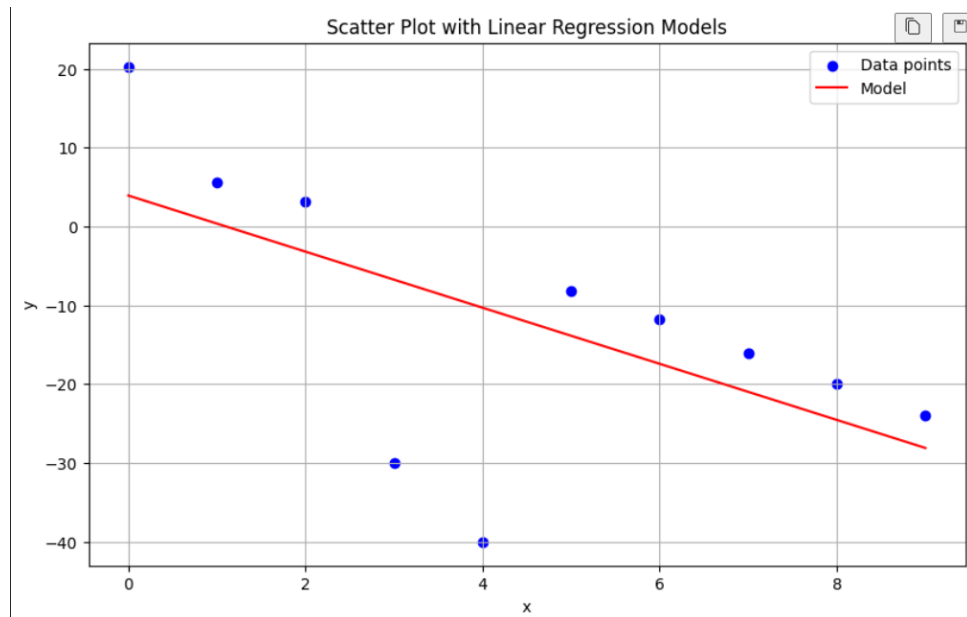


Figure 1: Scatter plot of data points with linear regression line.

1.3 Robust Estimator

Given Robust estimator is:

$$L(\theta, \beta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{(y_i - \hat{y}_i)^2}{(y_i - \hat{y}_i)^2 + \beta^2} \right)$$

Define the estimator:

```

1 x = np.array([0,1,2,3,4,5,6,7,8,9])
2 y_true = np.array([20.26, 5.61, 3.14, -30.00, -40.00, -8.13,
   ↪ -11.73, -16.08, -19.95, -24.03])
3
4 N = len(x)
5 y_hat1 = -4*x + 12
6 y_hat2 = -3.55*x + 3.91
7
8 def loss(a,b,alpha):
9
10     return (a-b)**2 / ((a-b)**2+ alpha**2)
11
12 def L(y_hat, y_true , beta ):
13     N = len(y_hat)
14     loss_total = 0
15     for i in range(N):
16         loss_total += loss(y_hat[i], y_true[i], beta)
17         #print(f"Loss for {i}th data point: {loss(y_hat[i],
   ↪ y_true[i], beta)}")
18     return loss_total / N
19
20

```

1.4 Test loss function for given β values

```
1 # Model1 -4x + 12 Loss
2 beta = (1, 10**-6, 10**3)
3
4 #beta = 1
5 L_1 = L(y_hat1, y_true, int(beta[0]))
6 print(f"Model1 Loss: {L_1}, beta = {beta[0]}")
7
8 #beta = 10^-6
9 L_2 = L(y_hat1, y_true, beta[1])
10 print(f"Model1 Loss: {L_2}, beta = {beta[1]}")
11
12 #beta = 10^3
13 L_3 = L(y_hat1, y_true, beta[2])
14 print(f"Model1 Loss: {L_3}, beta = {beta[2]}")
15
16 # Model2 -3.55x + 3.91 Loss
17
18 #beta = 1
19 L_1 = L(y_hat2, y_true, int(beta[0]))
20 print(f"Model2 Loss: {L_1}, beta = {beta[0]}")
21
22 #beta = 10^-6
23 L_2 = L(y_hat2, y_true, beta[1])
24 print(f"Model2 Loss: {L_2}, beta = {beta[1]}")
25
26 #beta = 10^3
27 L_3 = L(y_hat2, y_true, beta[2])
28 print(f"Model2 Loss: {L_3}, beta = {beta[2]}")
```

Results:

Model 1 ($y = -4x + 12$)

Model1 Loss: 0.435416262490386, beta = 1

Model1 Loss: 0.999999998258206, beta = 1e-06

Model1 Loss: 0.0002268287498440988, beta = 1000

Model 2 ($y = -3.51x + 3.91$)

Model2 Loss: 0.9728470518681676, beta = 1

Model2 Loss: 0.999999999999718, beta = 1e-06

Model2 Loss: 0.00018824684654645654, beta = 1000

1.5 Optimal β value

Optimal $\beta = 1000$

Justification:

Why $\beta = 1000$ is suitable:

$$L(\theta, \beta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{(y_i - \hat{y}_i)^2}{(y_i - \hat{y}_i)^2 + \beta^2} \right)$$

- In the numerator we have $(y_i - \hat{y}_i)^2$ while in denominator we have $(y_i - \hat{y}_i)^2 + \beta^2$
- By putting large enough β value we can make the denominator large enough so the errors from outliers doesn't impact the model loss so much.
- At $\beta = 10^{-6}$ the loss converges to 1, making it hard to choose between models and it does not mitigate the impact of the outliers since outliers cause larger errors.
- Both models show significant losses when $\beta = 1$ (e.g., Model 1: 0.4354, Model 2: 0.9728). So we should increase β value to mitigate the impact of the outliers
- $\beta = 1000$ keeps the loss in a sensitive (non-saturated) region, enabling clear comparison of models.
- Differences between models remain visible (Model 2 slightly better than Model 1).
- Provides a good trade-off between sensitivity to residuals and numerical stability.
- Increasing β furthermore will result in both losses converging into 0.

1.6 Optimal Model

- Both models are better when β is 1000 but Model 2 appears to have a lesser loss(0.00018825) compared to Model 1 loss(0.00022683).
- So we choose Model 2 $y = -3.51 * x + 3.91$
- It is also confirmed with the slope(-3.52) and intercept(3.92) values we achieved in the regression line equation.
- We can conclude that Model 2 $y = -3.51 * x + 3.91$ performs better when $\beta = 1000$ for the given dataset

1.7 Impact of the robust estimator on outliers

$$L(\theta, \beta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{(y_i - \hat{y}_i)^2}{(y_i - \hat{y}_i)^2 + \beta^2} \right)$$

- In the numerator we have $(y_i - \hat{y}_i)^2$ while in denominator we have $(y_i - \hat{y}_i)^2 + \beta^2$
- large denominator and small numerator will result in a small numerical value. In the robust estimator, the error term will be large for outliers because they lie far from the general data. By putting large enough β value we can make the denominator large enough so the errors from outliers doesn't impact the model loss so much
- Increasing β value too much will cause all errors to converge towards 0 which loses the credibility of the loss value to make decisions on the model. It also hides the presence of outliers.
- When β is small the denominator is almost equal to the numerator. So the error term is almost equal to the actual error. So outliers give large errors causing the total error to increase significantly.

1.8 Absolute Error instead of Squared Error

Absolute Error:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Loss function:

$$L(\theta, \beta) = \frac{1}{N} \sum_{i=1}^N \mu \frac{|y_i - \hat{y}_i|}{|y_i - \hat{y}_i| + \beta}$$

- Penalizes deviations linearly instead of quadratically.
- Prevents outliers from dominating the loss.
- The loss with β still scales smoothly.
- Maintains meaningful comparison between models.

2 Question 2

Filled Table:

Table 1: Comparison of MSE and BCE for different predictions

True y	Prediction $\hat{y}$	MSE	BCE
1	0.005	0.990025	0.004987
1	0.01	0.9801	0.009949
1	0.05	0.9025	0.048729
1	0.1	0.81	0.094824
1	0.2	0.64	0.178515
1	0.3	0.49	0.249672
1	0.4	0.36	0.306495
1	0.5	0.25	0.346574
1	0.6	0.16	0.366516
1	0.7	0.09	0.361192
1	0.8	0.04	0.321888
1	0.9	0.01	0.230259
1	1.0	0.0	0.0

```

1
2
3 #Mean Squared Error (MSE)
4 def MSE(y_hat, y_true):
5
6     N = len(y_hat)
7     errors = np.zeros(N)
8     loss_total = 0
9     for i in range(N):
10         errors[i] = (y_hat[i] - y_true)**2
11         print(f"Loss for {i}th data point: {(y_hat[i] -
12             ↪ y_true)**2}")
13         loss_total += (y_hat[i] - y_true)**2
14     return loss_total / N , errors
15
16 #Binary Cross Entropy (BCE)
17 def BCE(y_hat, y_true):
18     N = len(y_hat)
19     errors = np.zeros(N)
20     loss_total = 0
21     for i in range(N):
22         if y_hat[i] == 1:
23             loss_total += -(y_hat[i])*np.log(y_hat[i])
24             print(f"Loss for {i}th data point:
25                 ↪ {-(y_hat[i])*np.log(y_hat[i])}")
26         else:
27             errors[i] = -(1-y_hat[i])*np.log(1 - y_hat[i])

```

```

27         loss_total += -(1-y_hat[i])*np.log(1 - y_hat[i])
28     print(f"Loss for {i}th data point:
29         ↳ {-(1-y_hat[i])*np.log(1 - y_hat[i])}")
29     return loss_total / N , errors

1 MSE_value, error_MSE = MSE(y_hat, y_true)
2 BCE_value, error_BCE = BCE(y_hat, y_true)
3
4 y = 1
5 y_pred = np.array([0.005,0.01,0.05,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,
6     ↳ ,0.9,1.0])
7
8 # Plot
9 plt.figure(figsize=(8,5))
10 plt.plot(y_pred, error_MSE, 'o-', label='MSE')
11 plt.plot(y_pred, error_BCE, 's-', label='BCE')
12 plt.xlabel('Prediction  $\hat{y}$ ')
13 plt.ylabel('Loss')
14 plt.title('MSE and BCE vs Predictions')
15 plt.legend()
16 plt.grid(True)
17 plt.show()

```

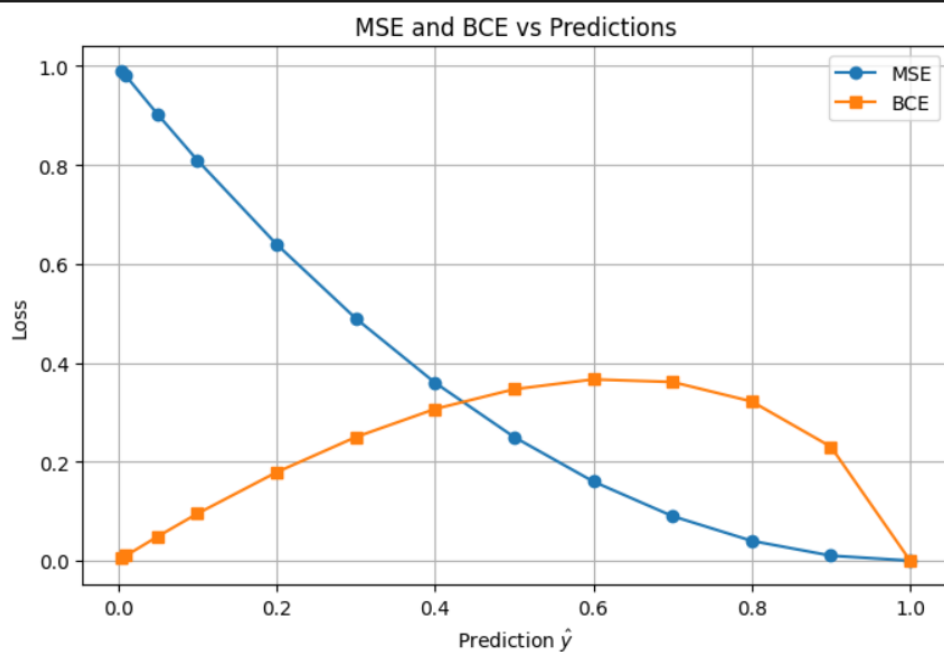


Figure 2: MSE,BCE vs predicted Y

For **Application 1**, the **Mean Squared Error (MSE)** is more suitable because the target variable is continuous. In continuous-variable tasks, the Binary Cross-Entropy (BCE) loss cannot be properly defined, as it assumes discrete labels. Therefore, MSE provides a natural and smooth measure of prediction error for continuous outcomes.

For **Application 2**, **BCE** is appropriate because the problem is of a classification or logistic regression type, where the target values are discrete (e.g., 0 or 1). In such cases, BCE effectively captures the difference between predicted probabilities and the true discrete labels, providing a meaningful gradient for optimization.

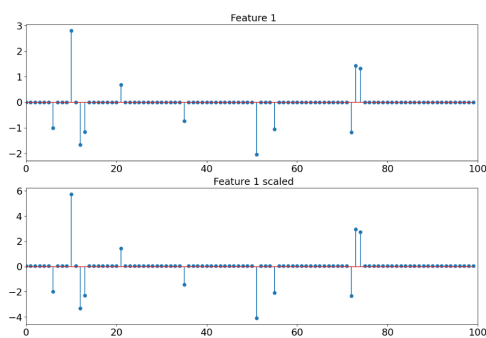
3 Question 3

```
1 feature_1 = sparse_signal
2 feature_2 = epsilon
3 feature_1 = feature_1.reshape(-1, 1)
4 feature_2 = feature_2.reshape(-1, 1)
```

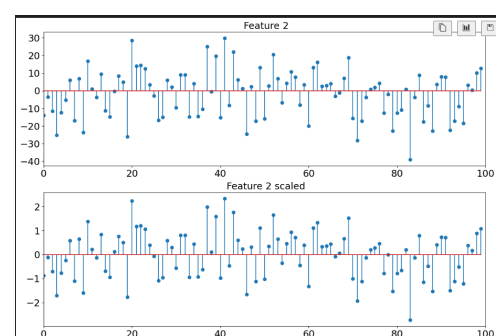
For scaling features I used scalers from scikit-learn.

Standard Scaling

```
1
2 scaler1 = StandardScaler()
3
4 feature_1_scaled = scaler1.fit_transform(feature_1)
5
6
7 feature_2_scaled = scaler1.fit_transform(feature_2)
8
```



(a) Feature 1



(b) Feature 2

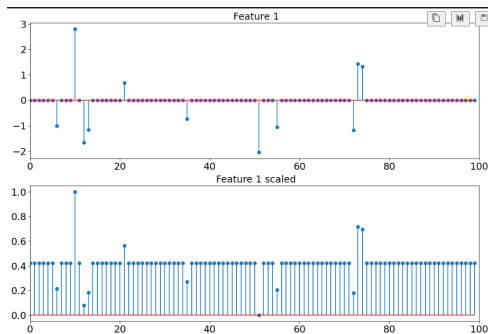
Figure 3: StandardScaler for features

Min-Max Scaling

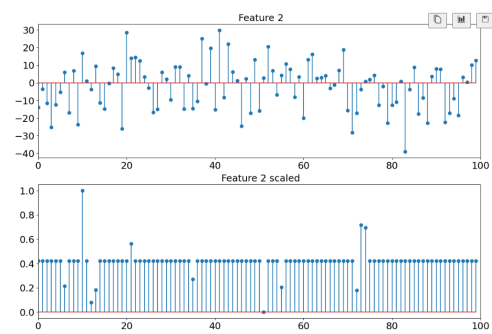
```

1
2 scaler2 = scaler2 = MinMaxScaler()
3
4 feature_1_scaled = scaler2.fit_transform(feature_1)
5
6 feature_2_scaled = scaler2.fit_transform(feature_1)
7
8

```



(a) Feature 1



(b) Feature 2

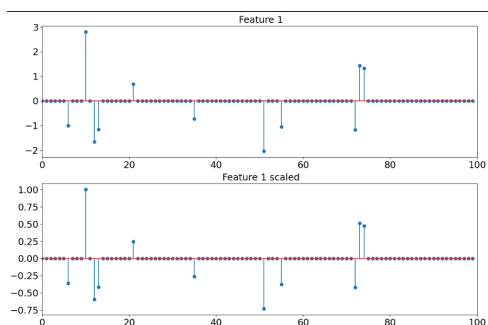
Figure 4: MinMaxScaler for features

Max-Abs Scaling

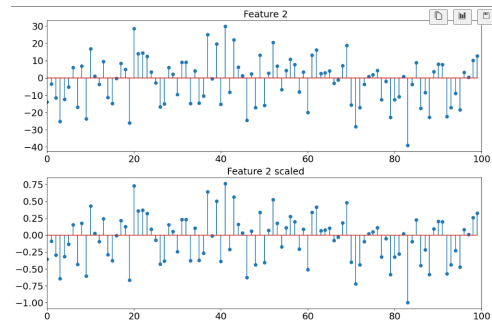
```

1
2 scaler3 = MaxAbsScaler()
3
4 feature_1_scaled = scaler3.fit_transform(feature_1)
5
6 feature_2_scaled = scaler3.fit_transform(feature_2)
7

```



(a) Feature 1



(b) Feature 2

Figure 5: MaxAbsScaler for features

Selecting the suitable scaler:

- Method (b) **Min-Max Scaler** is not optimal because it does not preserve the structure of the features.
- Both (a) **Standard Scaler** and (c) **Max-Abs Scaler** preserve the structure of the features.
- Method (c) **Max-Abs Scaler** always scales features between 0 and 1 while preserving their structure.
- Method (a) **Standard Scaler** depends on mean and standard deviation, so it does not guarantee fixed interval scaling.
- Therefore, the optimal scaling method is **Max-Abs Scaler**.

For more details please go to: [Github repository](#)